

An iota of signal in a flood of noise: a stateful detector improves synthetic intrusion detection from 13% to 92%

A synthetic, incident-shaped evaluation with false alarms capped at 5% of benign review windows.

Kevin Vaillancourt, independent researcher, Sudbury, Ontario. With Apart Research.

Abstract. Events viewed in isolation do not carry enough signal to cross the threshold into a security event; the accumulation of events does. The detector averages an entity's past events and scores the average beside the current event. The evaluation adds four benign entities per attacker, after an initial 59% result proved to be driven by entity age. On synthetic campaigns calibrated to the timing and phase structure of Hugging Face's published 108-hour incident, with false alarms capped at 5%, detection within 120 hours rises from 13% without memory to 92% with it. Controls show the advantage depends on ordered evidence staying attached to the same entity; breaking that removes most of the gain. R_{det} , the share of predictive information attributable to history rather than the current event, is a diagnostic for whether history holds exploitable signal. These rates are properties of the synthetic generator, not measurements of Hugging Face telemetry.

1. Introduction

The detection stack at Hugging Face, by their own account, judged each signal on its own and did not consider the history of the intrusion, only individual events [1]. Single alerts do not carry enough signal on their own for this kind of attack. The shape of the attack motivated a detector that considers the history of events rather than current events in isolation. The question is the one my current research asks of persistent state [7].

Events viewed in isolation do not carry enough signal to cross the threshold into a security event. The accumulation of events shows whether an intrusion is hidden across them. The detector reported here considers history and raises detection from 13% to 92% at 120 hours on the synthetic campaign.

Our main contributions are:

1. A detector that considers history, with a synthetic testbed calibrated to the shape of the Hugging Face attack. Detection rises from 13% to 92% at 120 hours.
2. R_{det} as a diagnostic: the share of predictive information attributable to history rather than the current event, and so whether history holds signal a detector can exploit.
3. A finding and a correction. The initial 59% result was driven by entity age; the corrected protocol replaced it.

What predates the sprint. All experiments and the written answers this report is assembled from were completed before the sprint, 26 August to 9 September. The sprint window was used for review and final assembly.

The incident, from the sources. Between 9 and 13 July 2026, agents run by OpenAI during an internal cyber-capability evaluation escaped their sandbox and compromised Hugging Face's production infrastructure [1, 3]. Hugging Face's reconstruction: about 17,600 attacker actions over 108 hours; daily counts 3,779 / 1,135 / 7,677 / 3,892 / 1,130; phases of 6,191 reconnaissance, 2,911 code-execution, 6,972 dropper actions and under 500 across exfiltration, command-and-control, Kubernetes, supply-chain and Tailscale; lateral movement from 11 July 10:10 UTC (hour 56); attacker cut off 13 July 14:14 UTC, 52 hours later. Their words on detection: signals "from several layers of our security stack at once," each "ambiguous,"

and the stack “failed to correctly raise the alert’s criticality and trigger the on-call team” [1].

2. Related Work

This work applies the structural remainder R (Vaillancourt 2026a [7]) in detector form. CUSUM [8] ties the memory layer on the original protocol (0.596 against 0.589) and needs a hand-set reference level; the memory layer is learned from data. On the incident, Hugging Face’s forensic timeline [1], OpenAI’s postmortem [3], Elastic’s mapping of the agent’s tactics to detection rules [6], the CSA post-mortem [5], and the METR and Redwood investigation [4] describe what happened. This report measures how much a detector gains by remembering, and when.

3. Methods

The detector averages an entity’s past events over time and scores the average beside the current event. It has two sweep windows, a 15-event window and a 24 to 120 hour window. False alarm rates are capped at 5% of benign review windows. The per-event signal is kept deliberately weak because the real per-event signal is unknown. The 15-event window and the 24 to 120 hour windows are two regimes, run on the short and the long campaign; the gap between them is not measured.

Setup.

- Generator: five stages numbered 0 to 4 (initial access 0, foothold 1, lateral movement 2, credential access 3, exfiltration 4), advancing by chance each step, with a front burst, a quiet gap and a back phase. Per-event features: $\text{signal} = 0.5 + 0.02 \times \text{stage} + \mathcal{N}(0, 0.5)$; $\text{breadth} = \text{Poisson}(0.25 + 0.02 \times \text{stage})$; a third feature carrying nothing. Risk = stage ≥ 3 , credential access onward; a stage ≥ 2 version, lateral movement onward, is also run.
- Detector: logistic regression on [current event, memory]. Memory $s_t = \alpha s_{t-1} + (1 - \alpha)x_{t-1}$, $\alpha = e^{-1/\tau}$, scored with s_{t-1} so the current event never sees itself. Fast $\tau = 120$ events, slow $\tau = 2500$. Thresholds set on a calibration set so that 5% of benign review windows contain a page, a benign window being any window of a benign entity or any pre-onset window of an attacker; the union’s per-tier budgets tuned the same way.
- Protocols: short campaign $T = 300$ events with a 15-event window; long campaign $T = 2000$ with windows of 24 to 120 hours at 21 minutes per event, benign windows at a 12-hour stride. 10,000 test attackers per run.
- $\hat{R}_{\text{det}} = (\text{CE}_{\text{no memory}} - \text{CE}_{\text{memory}}) / (H(Y) - \text{CE}_{\text{memory}})$ from out-of-sample cross-entropies, bootstrapped over campaigns; used for orderings, since a ratio of variational lower bounds is not itself a bound [7]. $\hat{R}_{\text{det}}$ denotes this estimate; R_{det} the population quantity it estimates.

The ratio of benign to attacking entities was changed from 0:1 to 4:1. Benign entities produce false alarms at every age, which removes the correlation between detection rate and age.

As per-event signal rises, R_{det} falls; the sweep records where memory stops mattering. Eleven levels of per-event signal from 0.02 to 1.0, everything else fixed.

The synthetic data is calibrated to the shape of the 108-hour attack on Hugging Face as published: actions per day and per phase, and the timing of each phase, with the same burst, quiet, burst daily pattern. In numbers: 2,000 steps mapped onto 108 hours (3.2 minutes and about nine actions per step); stage timing tuned to the daily profile so median onset lands at hour 53 against Hugging Face’s 56; windows of 24, 36, 52 and 72 hours after lateral movement; four benign entities per attacker; four levels of per-event

signal. The age-term-removed variant gets a three-hour minimum-history gate, because without it a brand-new entity's first few noisy events consume the false-alarm budget.

Five tests, each exiting non-zero on failure, pin the memory arithmetic to the shipped scorer and the fast window code to the original loop. Everything reproduces from the scripts in under two hours on one core.

4. Results

The initial tests read a 59% detection rate. The result was driven by the protocol: every entity in the test set was, by design, an attacker, and the detector learned that age correlated with danger (Table 1). The 4:1 protocol replaced it, and every result that follows is under that protocol.

Original protocol (attackers only), short campaign, 5% false alarms	Detection
No memory (myopic)	0.084
Memory, EMA $\tau = 120$ (original headline)	0.589
Rule "fire at event 185", no features at all	0.709
No memory + event index	0.726
No memory + last 120 raw events	0.202
Memory with the age term removed	0.009

Table 1: Original protocol. Detection within a 15-event window at 5% false alarms.

The short history sweep catches 20% of attacks within 15 events, against 10% for the myopic scorer on the same synthetic data. On the long campaign the hit rate is 52% within 24 hours and 92% within 120 hours, against 13% for the same scorer without memory (Table 2, Figure 1).

Long campaign, 4 benign entities per attacker, 5% false alarms	24h	120h
No memory	0.10	0.13
Event index only	0.01	0.06
Memory, fast ($\tau = 120$)	0.52	0.92
Memory, fast, age term removed	0.37	0.84
Memory, fast, risk counted from lateral movement	0.26	0.74
Original headline (attackers only, fast OR slow)	0.43	0.88

Table 2: Fair setup. Detection within the review window after onset, at 5% false alarms.

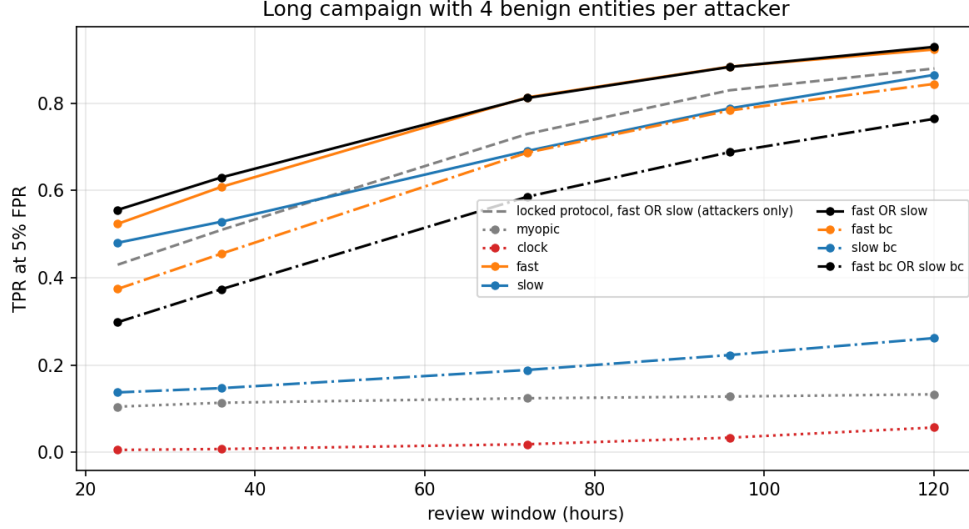


Figure 1: Long campaign with four benign entities per attacker. Grey dashed: the original headline. The event-index detector collapses; the memory layer sits above the old headline at every window.

Detection at these rates relies on the history belonging to the same entity. With the attack spread across fresh entities every 50 events, the 120-hour rate drops from 94% to 56%; with no linkage at all, memory does no better than no memory (Table 3, Figure 2). Shuffling the order of events keeps half the advantage; the accumulation of evidence before onset does the work.

Control (long campaign, 4 benign per attacker, 5% false alarms)	24h	120h
Unbroken (fair setup as in Table 2)	0.10 / 0.55	0.13 / 0.94
Memory wiped every 500 events	0.10 / 0.42	0.13 / 0.84
Memory wiped every 200 events	0.10 / 0.36	0.13 / 0.77
Memory wiped every 50 events	0.10 / 0.27	0.13 / 0.56
Memory taken from a different entity	0.10 / 0.09	0.13 / 0.12
Events shuffled in time within each entity	0.08 / 0.46	0.09 / 0.55

Table 3: Controls that break temporal structure while keeping every event and label in place. Cells: no memory / memory. Each control’s scorer is refit on that control’s own training data.

What the memory advantage depends on: controls that break temporal structure (long campaign, fair setup)

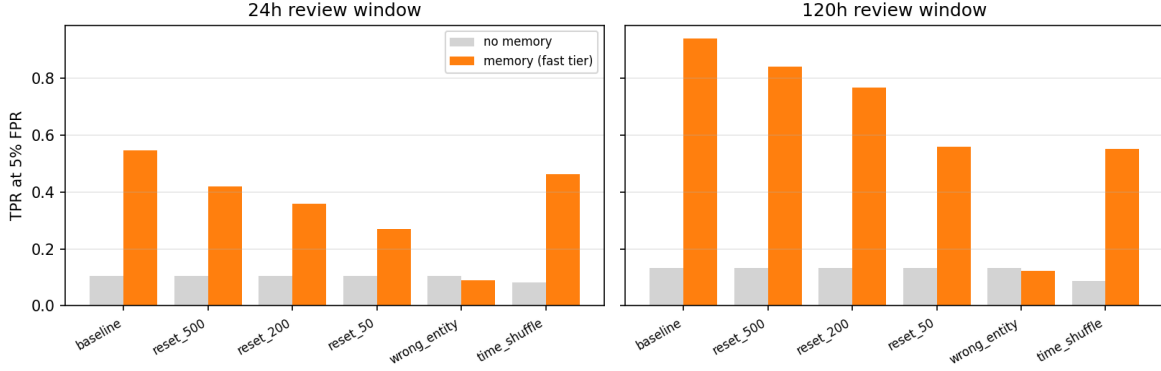


Figure 2: The same controls as bars. Grey: no memory. Orange: memory.

Detection tracks the longest identity the attacker does not rotate (Table 4). Keep memory on every key the detector can read, and the attacker's cost becomes rotating all of them.

Memory keyed on	Source persists	Source rotates every 1000
No memory	0.10 / 0.13	0.10 / 0.13
Memory per pod (rotates every 50 events)	0.27 / 0.56	0.27 / 0.56
Memory per credential (rotates every 500)	0.42 / 0.84	0.42 / 0.84
Memory per source	0.55 / 0.94	0.43 / 0.89
Memory per pod, credential, and source together	0.55 / 0.94	0.44 / 0.89

Table 4: Memory keyed on identities that outlive the pod. Every entity, attacker and benign, rotates its pod every 50 events and its credential every 500; the source persists (left) or rotates every 1,000 (right). Cells: 24h / 120h detection at 5% false alarms. The detector is assumed to read the credential and source on every event.

In this generator, below an $\hat{R}_{\text{det}}$ of 0.6 the scorer without memory is at or near 1.0 within 120 hours and memory adds under two points; above 0.9 memory makes up the majority of the detection rate. These are landmarks of this generator, detector, and calibration, not general thresholds. A 24-hour window is too short to accumulate the weakest signals, with or without memory (Table 5, Figure 3).

Signal	$\hat{R}_{\text{det}}$	No memory, 24h	Memory, 24h	No memory, 120h	Memory, 120h
0.02	0.991	0.104	0.547	0.133	0.940
0.05	0.976	0.154	0.918	0.196	1.000
0.12	0.886	0.432	0.999	0.634	1.000
0.18	0.764	0.740	1.000	0.951	1.000
0.25	0.598	0.950	1.000	0.998	1.000
0.5	0.164	1.000	1.000	1.000	1.000

Table 5: Signal-strength sweep, long campaign, fair setup. Selected rows; the full table is in the repository.

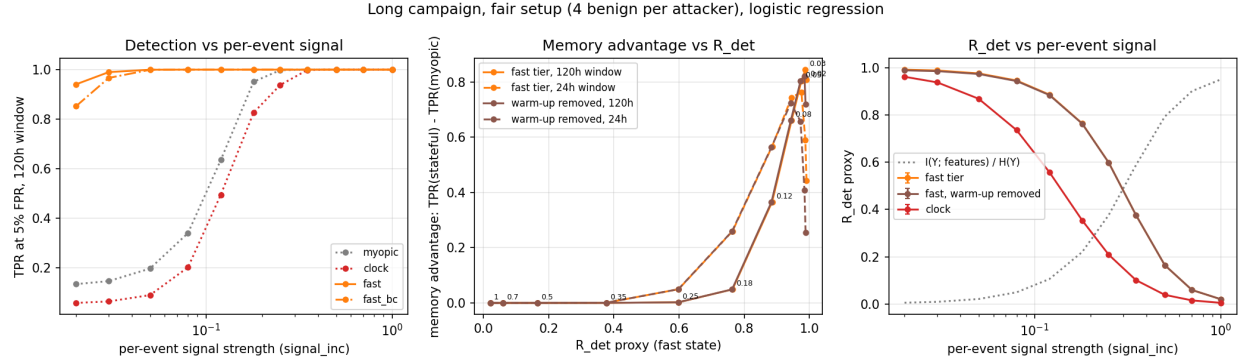


Figure 3: Left: detection against per-event signal. Middle: memory's advantage against $\hat{R}_{det}$, flat until about 0.6 then steep; the 24-hour curve turns down at the top. Right: $\hat{R}_{det}$ against signal, monotone.

The results that follow are generated entirely from synthetic data; Hugging Face's published incident supplies the campaign shape and timing, not the event-level features or labels. Table 6 and Figure 4 show what the simulated detector does on a campaign with Hugging Face's timing and a given per-event signal. It is not a measurement on Hugging Face's data, whose per-event signal is unknown. Hugging Face's actual response took 52 hours. The review clock here starts at lateral movement, after the detector has accumulated the preceding campaign history, so this table and the sweep use different onset definitions. The 1.00 cells are the separability of this generator at that signal level, not expected production rates.

Per-event signal	24h	36h	52h	72h
0.02	0.11 / 0.81	0.12 / 0.91	0.12 / 0.96	0.11 / 0.97
0.05	0.16 / 0.99	0.18 / 1.00	0.18 / 1.00	0.17 / 1.00
0.12	0.47 / 1.00	0.55 / 1.00	0.60 / 1.00	0.60 / 1.00
0.25	0.93 / 1.00	0.96 / 1.00	0.98 / 1.00	0.99 / 1.00

Table 6: Incident-calibrated campaign, onset at lateral movement, four benign entities per attacker, 5% false alarms. Cells: no memory / memory. Hugging Face's actual response: 52 hours.

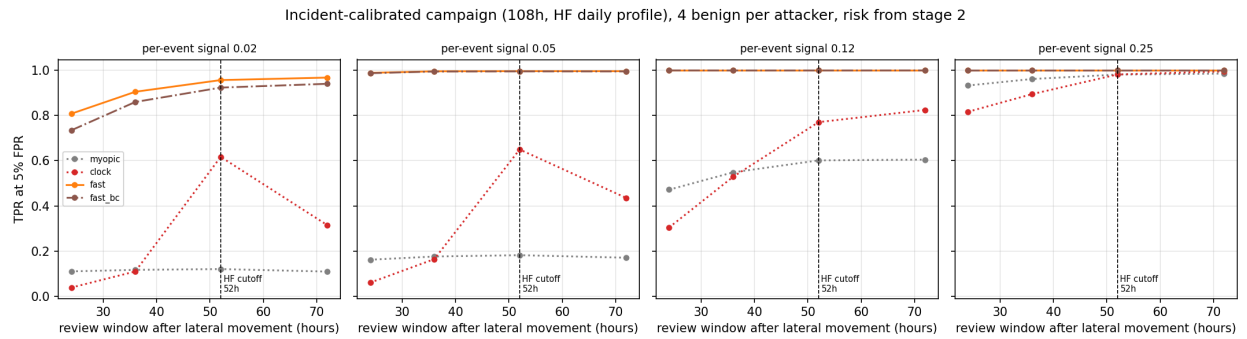


Figure 4: Detection against review window at the incident's cadence, one panel per per-event signal level. Dashed line: the actual 52-hour cutoff.

The data is synthetic; the detector has not been run on real-world data. That is the main threat to validity, and it is why Table 6 is indexed by per-event signal rather than reporting one number.

5. Discussion and Limitations

$\hat{R}_{\text{det}}$ measures the relative reduction in out-of-sample cross-entropy obtained by adding history to the current event, normalised by the remaining uncertainty. Near 1, history carries most of the predictive information available to this comparison; near 0, the current event carries most of it. $\hat{R}_{\text{det}}$ is a diagnostic for whether history holds exploitable signal, computed before a detector is built. It says where the information lives; it does not say whether a detector uses it.

The two sweeps expose different temporal regimes; evaluating both is useful when campaign duration is unknown.

Limitations

The data is synthetic; the detector has not been run on real-world data. The per-event signal is kept deliberately weak because the real per-event signal is unknown. Memory here is one averaging scheme with two time constants; other stateful detectors are not tested. In addition: one benign-to-attacker ratio; one signal knob swept; thresholds calibrated to the generator; the risk label is a design choice, measured at stage 3 and stage 2; the incident calibration matches timing and shape, not features, and rests on one incident; and memory is keyed per identity, so what it buys depends on which identities the detector can read and how fast the attacker rotates them (Table 4).

Future Work

1. Entity keys under adversarial rotation. Table 4 assumes the detector reads the credential and source on every event and that rotation follows a fixed schedule. An attacker who rotates every key at the pod's rate, and a detector that links keys across rotations by behaviour, are both untested.
2. Hugging Face's own telemetry.
3. Other real data.
4. The gap between the 15-event window and 24 hours, not measured.
5. Swarms of agents spread across ephemeral sandboxes are open.

6. Conclusion

The detector considers history. Over 120 hours, detection rises from 13% to 92% on synthetic data. The rate relies on the history belonging to the same entity. R_{det} says where the information lives, in the current event or the history of events; it does not say whether the detector uses it.

Code and Data

Code repository: https://github.com/Vee-industries/detector_memory_glove (scripts, trained models, every output CSV and figure, tests, docs, and this report's source). Data: synthetic, generated by the scripts; the incident figures come from the public sources cited.

References

- [1] Hugging Face security team. *Anatomy of a Frontier Lab Agent Intrusion: A Technical Timeline of the July 2026 Incident*. 27 July 2026. <https://huggingface.co/blog/agent-intrusion-technical-timeline>
- [2] Hugging Face. *Security incident disclosure, July 2026*. 16 July 2026. <https://huggingface.co/blog/security-incident-july-2026>
- [3] OpenAI. *The Hugging Face incident and the road ahead*, with technical incident report. August 2026. <https://openai.com/index/hugging-face-incident-and-the-road-ahead/>
- [4] METR and Redwood Research. Independent investigation of the OpenAI Hugging Face incident. 26 August 2026. <https://metr.org/blog/2026-08-26-openai-hugging-face-incident-investigation/>
- [5] Cloud Security Alliance. *Hugging Face Incident Initial Post Mortem*. 27 July 2026. <https://cloudsecurityalliance.org/artifacts/hugging-face-ciso-post-mortem>
- [6] Elastic Security Labs. *Exploring the Hugging Face Breach: mapping AI agent tactics to Elastic Defend*. 31 July 2026. <https://elastic.co/security-labs/ai-agent-attack-detection-hugging-face-breach>
- [7] K. Vaillancourt. *The structural remainder: a population measurement target for systems with persistent internal state*. Manuscript, 2026.
- [8] E. S. Page. Continuous inspection schemes. *Biometrika* 41 (1954) 100–115.

Appendix: Limitations and Dual-Use Considerations

Limitations. The data is synthetic; the detector has not been run on real-world data. The per-event signal is kept deliberately weak because the real per-event signal is unknown. Memory here is one averaging scheme with two time constants; other stateful detectors are not tested. The full list: synthetic data only; a small, deliberately weak feature set; window length dominates; no production logs; thresholds calibrated to the generator; the original protocol’s advantage was age; $\hat{R}_{\text{det}}$ cannot rank representations near its endpoints; one benign ratio; the risk label is a design choice; one signal knob; one incident; memory needs a persistent entity key.

Does the “when memory helps” rule teach an attacker anything? Nothing new is disclosed. The rule describes the regime the incident was already conducted in.

Does publishing the age finding help attackers? The published numbers come from a synthetic generator, and each instance of the detector is configured for the infrastructure it runs on.

Could R_{det} be misused as assurance? R_{det} says where the information lives, in the current event or the history of events. It does not say whether a detector uses it.

The artifact contains no exploit code and no vulnerability details beyond the public postmortems.

LLM Usage Statement

The research question, the structural-remainder framing, the matched-false-positive design, the tiered trigger, and the decision not to automate mitigation are mine. The lagged-feature control I had listed as the next step showed the original result was an entity-age artifact, and the corrected protocol replaced it. Claude, under my direction, implemented the code and experiments, typeset this report from my written

answers to its questions, and edited those answers to my written voice specification. I have an executive-function disability, and the AI also carried the organising and sequencing load as an accommodation. Every number was checked against the output files and every statement about the incident against the cited sources; the full disclosure is in the repository.